

Aula 03 — Objetos, referências e identidade

Quando usamos uma variável para trabalhar com um objeto, o que essa variável realmente representa?

Retomando o Laboratório 02

```
ItemPedido item = new ItemPedido();
```

Onde está o objeto?

O que `item` representa?

Três partes, três papéis

```
ItemPedido item = new ItemPedido();
```

ItemPedido

tipo da variável

item

variável que mantém uma referência

new ItemPedido() cria um novo objeto.

Valores simples

```
int a = 10;  
int b = a;
```

O que foi copiado para **b** ?

A cópia é independente

MAIN.JAVA

```
int a = 10;  
int b = a;  
b = 5;  
System.out.println(a);
```

CONSOLE

10

Alterar `b` não altera o valor armazenado em `a`.

Agora, um tipo de classe

```
ItemPedido a = new ItemPedido();  
ItemPedido b = a;
```

O que foi copiado para **b** ?

Atribuição copia o valor

TIPO SIMPLES

```
int b = a;
```

O valor é um número.

TIPO DE CLASSE

```
ItemPedido b = a;
```

O valor é uma referência.

O objeto não foi copiado.

CONCEITO-CHAVE

Modelo central

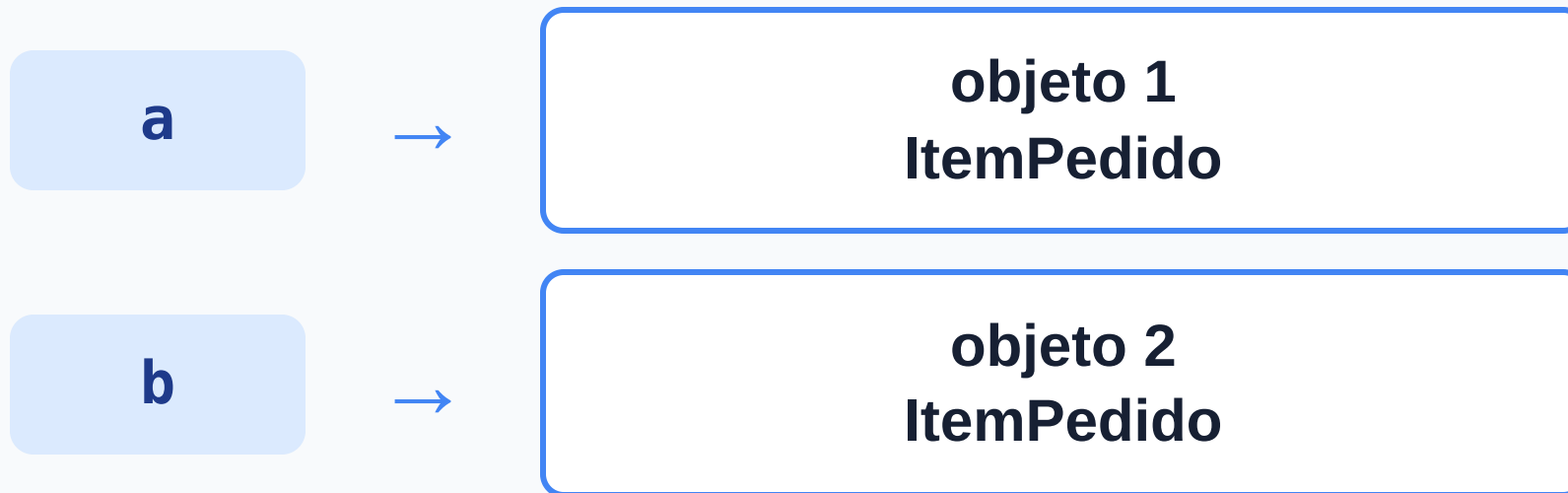
**A variável não é o objeto.
Ela mantém uma referência que permite acessar o
objeto.**

Dois usos de new

```
ItemPedido a = new ItemPedido();  
ItemPedido b = new ItemPedido();
```

- Quantas variáveis?
- Quantos objetos?

Dois usos de new



Dois new → dois objetos

Cada expressão `new ItemPedido()` cria um novo objeto.

Os objetos são distintos, mesmo sendo da mesma classe.

Copiando uma referência

```
ItemPedido a = new ItemPedido();  
ItemPedido b = a;
```

- Quantas variáveis?
- Quantos objetos?

Duas referências, um objeto



CONCEITO-CHAVE

Uma referência também é um valor

Duas variáveis podem manter referências que permitem acesso ao mesmo objeto.

ATIVIDADE

Preveja antes de executar

```
ItemPedido a = new ItemPedido();  
ItemPedido b = a;  
  
a.quantidade = 2;  
b.quantidade = 7;  
  
System.out.println(a.quantidade);
```

O que será apresentado? Por quê?

Execução simulada

MAIN.JAVA

```
ItemPedido a = new ItemPedido();  
ItemPedido b = a;  
a.quantidade = 2;  
b.quantidade = 7;  
  
System.out.println(a.quantidade);
```

CONSOLE

7

O estado pertence ao objeto



A alteração ocorreu no único objeto acessado por **a** e **b**.

Dois objetos com os mesmos valores

```
ItemPedido a = new ItemPedido();  
a.descricao = "Teclado";  
a.quantidade = 2;  
  
ItemPedido b = new ItemPedido();  
b.descricao = "Teclado";  
b.quantidade = 2;
```

Estado × identidade

objeto 1

descrição = "Teclado"
quantidade = 2

objeto 2

descrição = "Teclado"
quantidade = 2

- Possuem o mesmo estado?
- São o mesmo objeto?

Mesmo estado, identidades diferentes

Estado

valores que o objeto possui naquele momento

Identidade

se estamos falando do mesmo objeto ou de objetos diferentes

Dois objetos podem ter o mesmo estado e continuar distintos.

Primeiro cenário

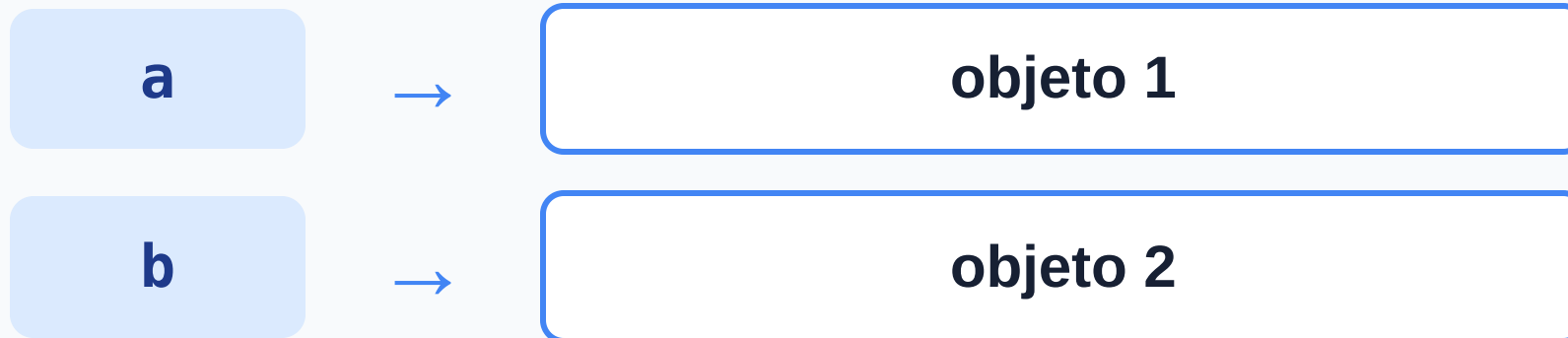
```
ItemPedido a = new ItemPedido();  
ItemPedido b = new ItemPedido();
```

```
a == b
```

true ou false ? Por quê?

Dois objetos distintos

false



Segundo cenário

```
ItemPedido a = new ItemPedido();  
ItemPedido b = a;
```

```
a == b
```

true ou false ? Por quê?

O mesmo objeto

true



== e referências

Neste contexto, `==` verifica se duas referências correspondem ao mesmo objeto.

Atividade principal

```
ItemPedido a = new ItemPedido();  
a.quantidade = 2;  
  
ItemPedido b = a;  
  
ItemPedido c = new ItemPedido();  
c.quantidade = 2;  
  
b.quantidade = 4;
```

ATIVIDADE

Façam a previsão

1. Quantos objetos foram criados?
2. Quais variáveis compartilham o mesmo objeto?
3. Quais quantidades serão observadas por `a` , `b` e `c` ?

Primeiro: quantos objetos?



Depois: quais valores?

MAIN.JAVA — CONSULTAS

```
System.out.println(a.quantidade);  
System.out.println(b.quantidade);  
System.out.println(c.quantidade);
```

CONSOLE

4

4

2

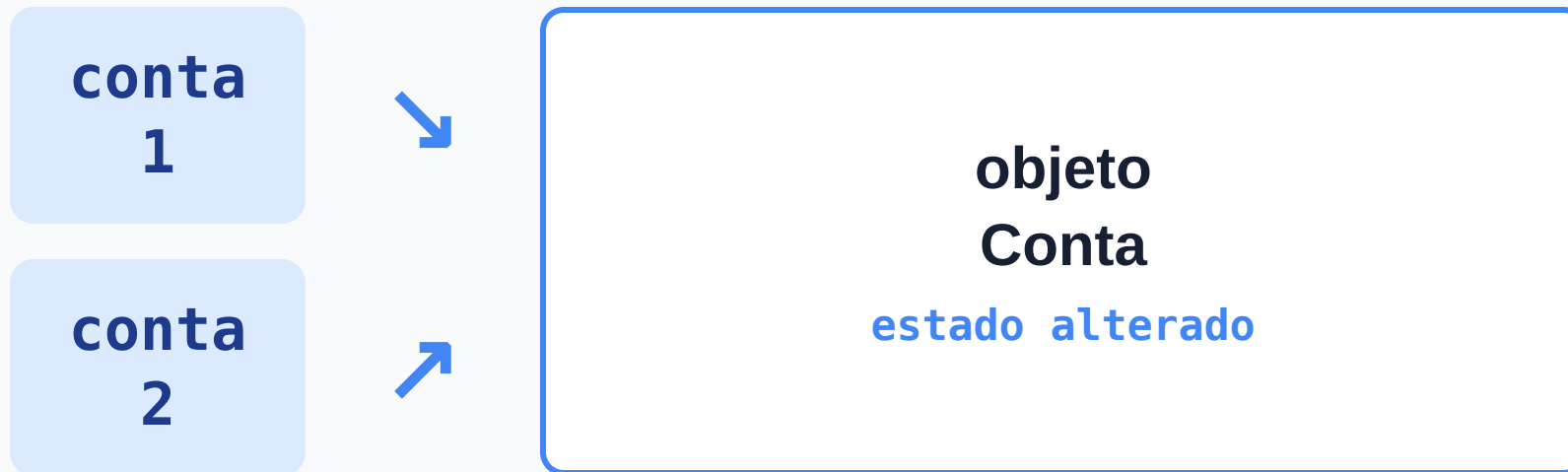
`b.quantidade = 4` alterou o estado do objeto acessado por `a` e `b`.

Transferindo o modelo

```
Conta conta1 = new Conta();  
Conta conta2 = conta1;
```

Se uma operação usando `conta2` modificar o objeto, o que será observado ao acessá-lo por `conta1` ?

O domínio mudou. O modelo não.



A alteração é observável por ambas as referências porque existe um único objeto.

Síntese — criação e acesso

`new` cria um objeto

→

a variável mantém uma referência

→

a referência permite acessar o objeto

Atribuir uma referência a outra variável não copia o objeto.

Síntese — estado e identidade

Estado

pode ser observado e alterado por diferentes referências ao mesmo objeto

Identidade

objetos diferentes continuam distintos mesmo com valores iguais

== foi usado aqui para verificar identidade.

Um problema fica em aberto

```
item.quantidade = -200;
```

Na estrutura atual, quem possui uma referência pode alterar diretamente o estado.

Se o objeto é responsável pelo próprio estado, qualquer parte do programa deveria poder alterá-lo diretamente?
